HOW FAR CAN MY E-SCOOTER REALLY GO?

REVERSE ENGINEERING REAL-TIME BATTERY AND SPEED DATA WITH ESP32, BLUETOOTH LOW ENERGY (BLE), AND RS485

BY MINH TRAN
ELECTRICAL ENGINEERING STUDENT
UNIVERSITY AT BUFFALO

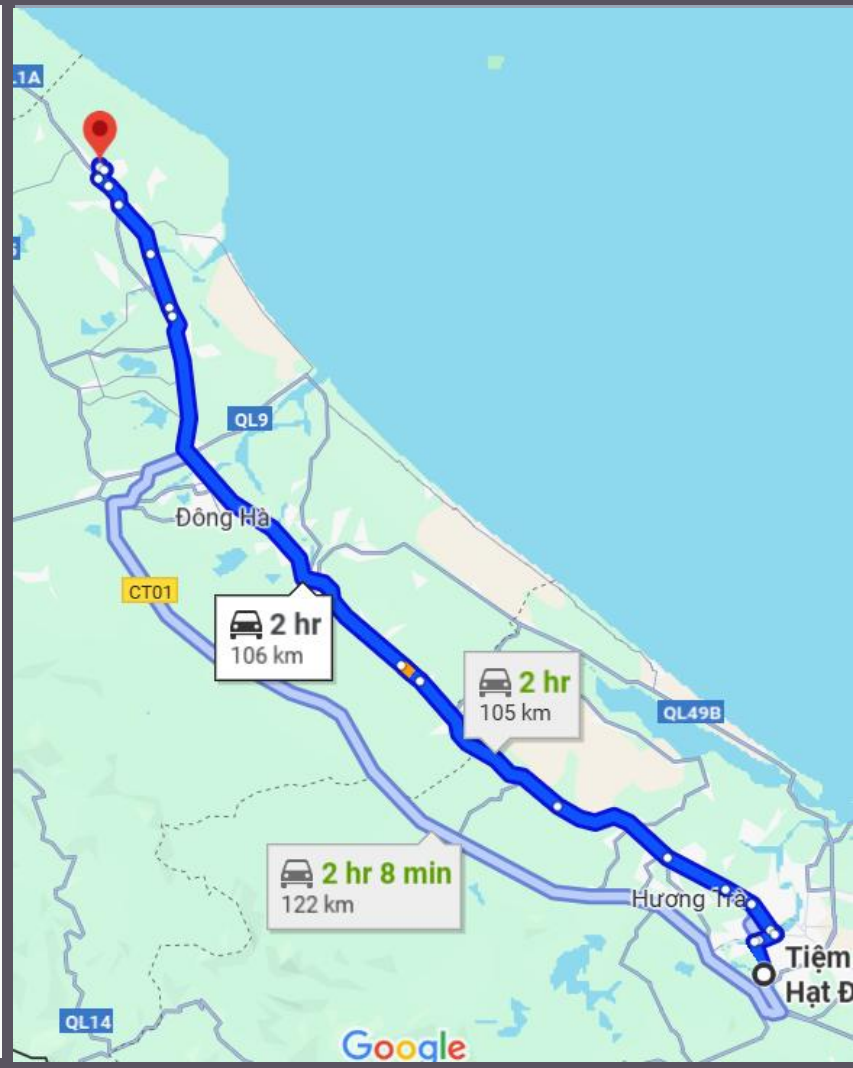
Problem:

My VinFast Feliz e-scooter displayed a full-charge range of about **150 km**, but in real riding conditions it only reached about **100-132 km**.

That raised a simple question:

Why was the displayed range so different from the real-world range?

To answer that, I needed to look beyond static battery percentage and capture what the scooter was actually doing in real time.



Speed:	18.7 km/h
Remaining Dist:	113.3 km
SoC (Wh):	70.6%
Remaining Time:	6h 3m
SoC (Ah):	68%
Current:	-6.483 A
Voltage:	77.15 V
Capacity (Wh):	4292.0 Wh
Capacity (Ah):	58.000 Ah
Remaining (Wh):	3031.2 Wh
Remaining (Ah):	39.290 Ah

Figure 1

Speed:	7.3 km/h
Remaining Dist:	17.5 km
SoC (Wh):	70.5%
Remaining Time:	2h 23m
SoC (Ah):	68%
Current:	-16.437 A
Voltage:	76.99 V
Capacity (Wh):	4292.0 Wh
Capacity (Ah):	58.000 Ah
Remaining (Wh):	3027.7 Wh
Remaining (Ah):	39.326 Ah

Figure 2

Range isn't fixed.

It depends on:

- Real-time current draw
- Slope, traffic, load, acceleration...

Figure 1 shows stable speed -> lower power -> higher distance

Figure 2 shows acceleration -> higher power -> lower distance

→ Static % State of Charge (SoC) isn't enough!

RS485 Modbus RTU V1.0

BLE Notify

Hall pulse counting

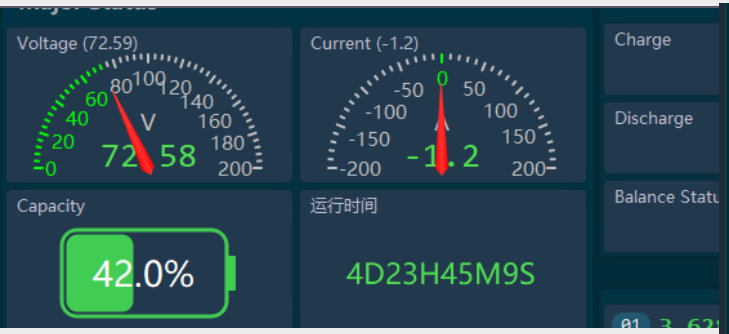
My solution:

A custom system that tracks live data:

- Voltage, Current, SoC (via RS485)
- Speed (via Hall sensor)
- Processed on ESP32
- Sent via BLE to Android application

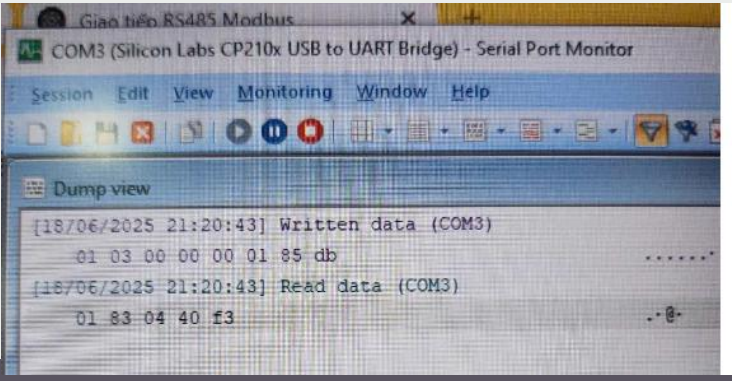
**RS485: a wired serial communication bus used to read battery telemetry data.*
BLE (Bluetooth Low Energy): a wireless protocol used to send live data to the Android app.
ESP32: the microcontroller that reads, processes, and transmits the scooter data.

Image by xevinfastvn.com; jkbms.com; espressif; freeiconspng.com



Official app method -> **success**

```
[19:56:37,559] 55
[19:56:37,597] AA EB 90 02 00 2C 0E 2C 0E 2E 0E 2C 0E
2E 0E 2C 0E 2C 0E 2E 0E 2E 0E 2C 0E 2C 0E 2E 0E 2E 0E
2C 0E 30 0E 2E 0E 2E 0E 2E 0E 2E 0E 2E 0E 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 FF FF 0F 00 2D 0E 05 00 0E 09
[19:56:37,598] 9B 00 98 00 97 00 97 00 98 00 99 00 95
00 96 00 99 00 98 00 99 00 9D 00 9E 00 9C 00 9C 00 9D
00 9D 00 9E 00 9E 00 9C 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 00 00 00 00 00 00 51 01 00
00 00
[19:56:37,598] 00 89 1B 01 00 5B 6A 01 00 5D FB FF FF
3B 01 38 01 00 00 00 00 00 00 2A CB 6B 00 00 E8 FD
00 00 00 00 00 00 BB D3 00 00 64 00 00 00 05 94 06 00
01 01 00 00 00 00 00 00 00
[19:56:37,598] 00 00 00 00 00 00 00 FF 00 01 00 00 00
AC 23 00 00 0D 00 AB 76 3F 40 00 00 00 00 5A 1C 00 00
00 00 01 01 0A 01 01 00 9C 84 40 00 00 00 00 00 51 01
30 F8 30 F8 9B 23 76 D0 4D 0A B8 00 00 00 80 51 01 00
00 01
[19:56:37,599] 00 01 00 00 00 00 00 00 00 00 00 FE FF
7F DC 0F 01 00 80 0B 01 00 00 CB
[19:56:37,599] 01 10 16 20 00 01 04 4B
```



Datasheet method **failed!**

📦 RS485 Reverse Engineering:

No reliable datasheet was available

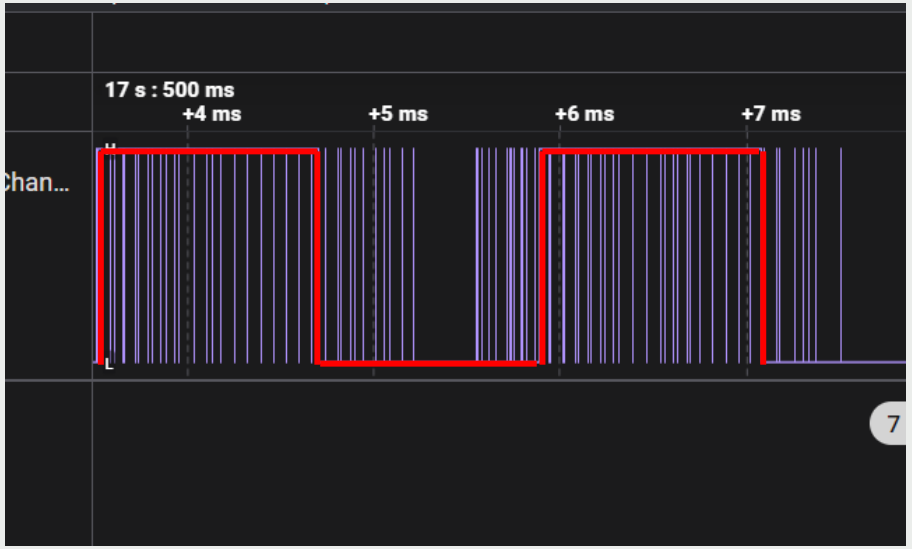
Sniffed Modbus packets from the official app

Parsed the data frame manually in code

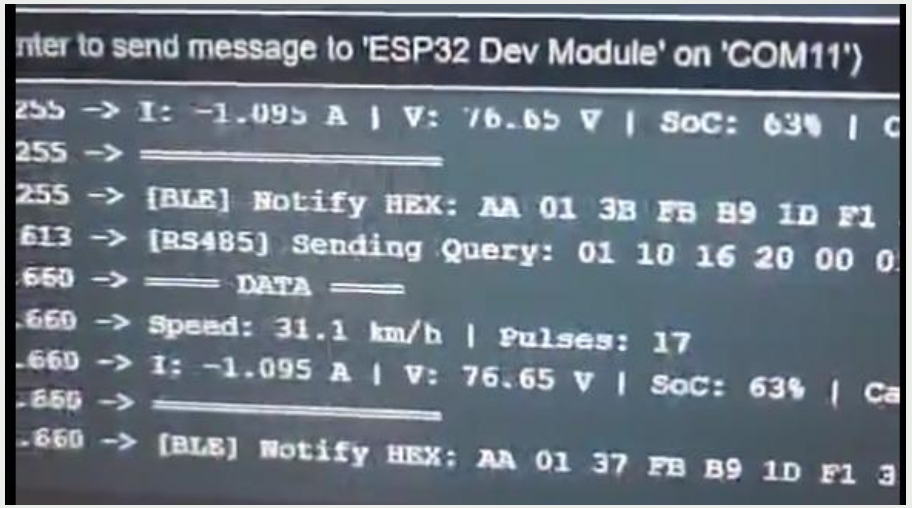
Decoded voltage, current, and capacity values

Modbus RTU: a **frame-based protocol used over RS485 to exchange battery data.*

JiKong actually provided a datasheet call RS485 Modbus RTU V1.0, but I couldn't use it to acquire data. Access here: http://www.jk-bms.com/en/Upload/2023-12-05/1559153362.pdf



Noisy raw signal



Filtered & Converted

🌀 Speed tracking:

- Used Hall encoder on wheel
- Sampled at 1ms, filtered via pulse duration

🧪 Tested signal & frame manually via Serial

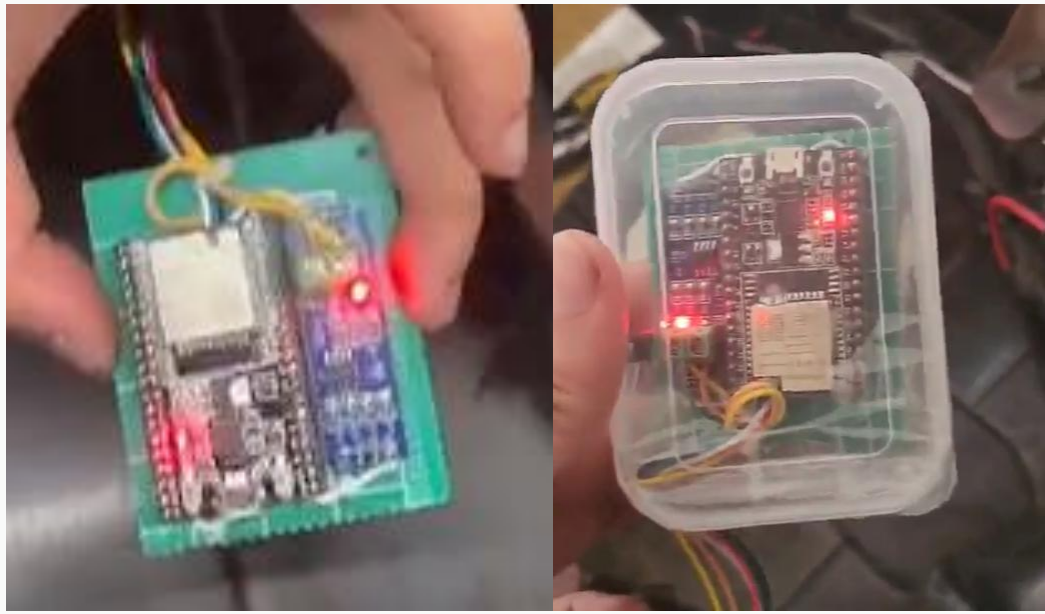
A compact hardware setup for stable RS485 communication:

MAX485 + ESP32 on PCB

Querying BMS every 400ms

Stable signal verified over UART + BLE

Fully tested with live powertrain data

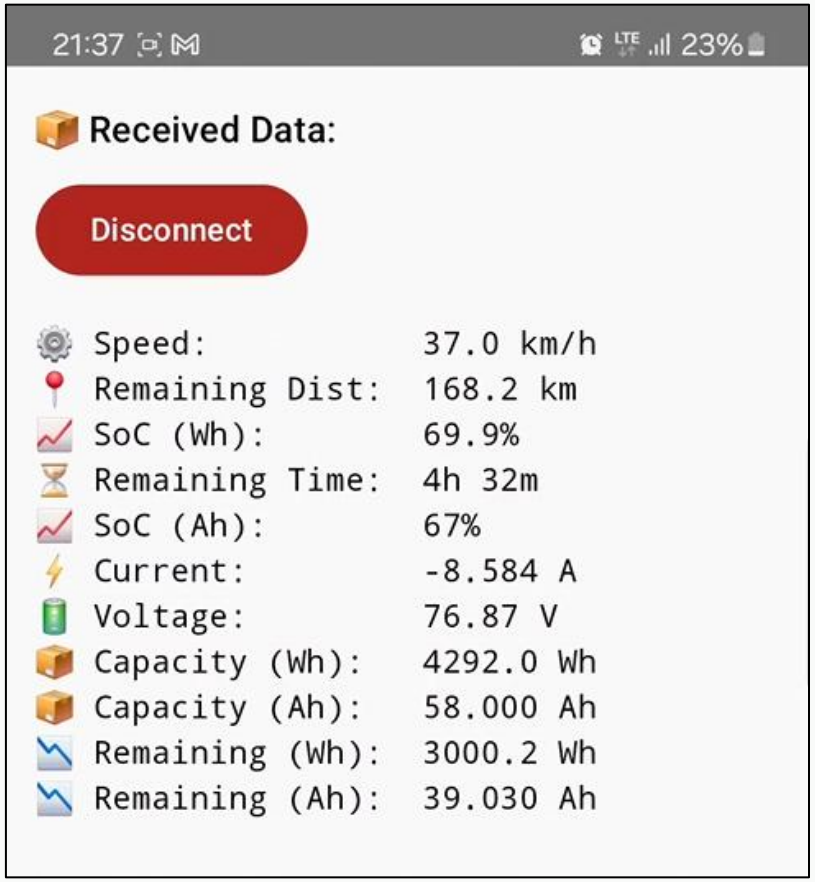


RS485 & ESP32 on PCB

📡 BLE Notify:

- ESP32 sends data every 400ms
- Android app decodes and displays:

→ Voltage, Speed, SoC in terms of Ampere-hour / Watt-hour (Ah / Wh)



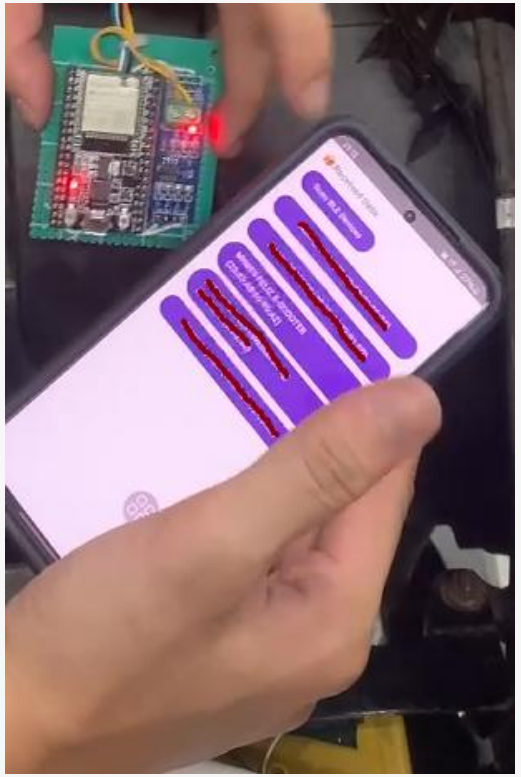
Results:

Built a real-time dashboard for scooter telemetry

Displayed live battery and speed data on an Android app

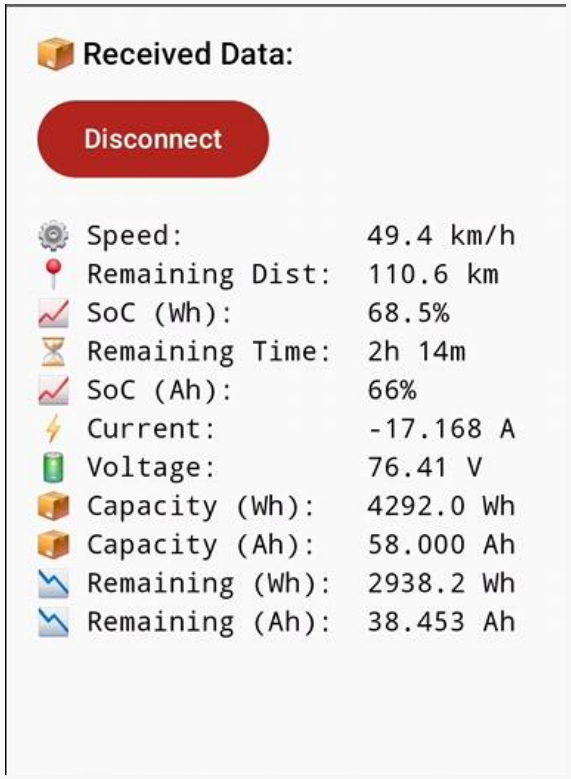
Improved range understanding under real riding conditions

Verified wireless data transfer through Bluetooth Low Energy (BLE)



BLE connection

Realtime log



App display

Lessons Learned:

Reverse engineering the RS485 communication bus required repeated testing and validation.

Static State of Charge (SoC) alone could not estimate real range accurately.

Software filtering was necessary to stabilize noisy sensor signals.

Bluetooth Low Energy (BLE) integration made live monitoring easier.

Hardware, firmware, and app data had to work together to make the system useful.

Conclusion

So, how far can my e-scooter really go?

Battery percentage alone cannot predict real-world range accurately.

Real range is influenced by live battery behavior, speed, rider input, and road conditions. By decoding the scooter's **RS485** battery data and combining it with real-time speed sensing, I created a system that estimates range more realistically in real-world use.

THANK YOU

MINH QUANG TRAN

Electrical Engineering Student

University at Buffalo

(+1) 585-414-4188

mtranqua@buffalo.edu